

CHAPTER – 4

ANALYZING FUNCTIONAL DEPENDENCIES, REDUNDANCY AND DATA ANOMALIES

4.1 INTRODUCTION:

The **Titan Eye Eyewear E-Commerce Database System** is designed to efficiently manage customers, eyewear products, categories, brands, orders, order items, payments, shipments, and administrators.

Functional dependencies help identify the relationships between attributes in each table. Redundancy analysis and data anomaly detection help maintain data consistency and reduce unnecessary duplication. A well-designed database improves data integrity, reduces redundancy, and supports efficient management of the Titan Eye Eyewear e-commerce system.

1. CUSTOMER TABLE:

Functional Dependency

Customer_ID → Customer_Name, Email, Phone_Number, Address, Password

Redundancy Analysis

Customer information is stored only once in the Customer table. Other tables use **Customer_ID** to identify the customer instead of repeating customer details.

Data Anomalies

- **Insert Anomaly:** A new customer can be added without placing an order.
- **Update Anomaly:** Updating a customer's phone number or address requires changes only in the Customer table.
- **Delete Anomaly:** Deleting an order will not remove the customer's information.

2. CATEGORY TABLE:

Functional Dependency

Category_ID $\rightarrow$ Category_Name

Redundancy Analysis

Each eyewear category is stored only once. The Product table refers to the category using **Category_ID**, avoiding repeated category names.

Data Anomalies

- **Insert Anomaly:** A new category can be added without adding a product.
- **Update Anomaly:** Renaming a category requires updating only one record.
- **Delete Anomaly:** Deleting a product does not delete the category.

3. BRAND TABLE:

Functional Dependency

Brand_ID $\rightarrow$ Brand_Name

Redundancy Analysis

Brand information is stored only once in the Brand table. Products refer to brands using **Brand_ID**, preventing repeated brand names.

Data Anomalies

- **Insert Anomaly:** A new brand can be added before adding its products.
- **Update Anomaly:** Changing a brand name requires updating only one record.
- **Delete Anomaly:** Deleting a product does not delete the brand information.

4. PRODUCT TABLE:

Functional Dependency

Product_ID → Product_Name, Price, Stock, Category_ID, Brand_ID, Frame_Color, Lens_Type

Redundancy Analysis

All eyewear product details are stored in the Product table. The Product table uses **Category_ID** and **Brand_ID** as foreign keys instead of repeatedly storing category and brand names. Order Item stores **Product_ID** instead of repeating complete product information.

Data Anomalies

- **Insert Anomaly:** A new eyewear product can be added before receiving any orders.
- **Update Anomaly:** Product details such as price, stock, frame color, or lens type can be updated in one place.
- **Delete Anomaly:** Deleting an order or order item does not delete the product information.

5. ORDER TABLE:

Functional Dependency

Order_ID → Customer_ID, Order_Date, Total_Amount, Order_Status

Redundancy Analysis

The Order table stores only order-related information. Customer details are referenced using **Customer_ID**, avoiding repeated customer information.

Data Anomalies

- **Insert Anomaly:** A new order can be created for an existing customer.
- **Update Anomaly:** The order status can be updated without affecting customer or product information.
- **Delete Anomaly:** Deleting an order does not delete the customer or product information.

6. ORDER ITEM TABLE:

Functional Dependency

Order_Item_ID $\rightarrow$ Order_ID, Product_ID, Quantity, Price

Redundancy Analysis

The Order Item table connects orders and products. It avoids storing multiple product details directly in the Order table.

Data Anomalies

- **Insert Anomaly:** A product can be added to an order independently.
- **Update Anomaly:** The quantity of a product can be changed for a particular order item.
- **Delete Anomaly:** Removing an order item does not delete the product from the Product table.

7. PAYMENT TABLE:

Functional Dependency

Payment_ID $\rightarrow$ Order_ID, Payment_Method, Payment_Status

Redundancy Analysis

Payment details are stored separately from the Order table. This prevents repeated storage of payment method and payment status information.

Data Anomalies

- **Insert Anomaly:** Payment details can be recorded after an order is created.
- **Update Anomaly:** Payment status can be updated independently.
- **Delete Anomaly:** Deleting a payment record does not delete the related order.

8. SHIPMENT TABLE

Functional Dependency

Shipment_ID → Order_ID, Tracking_Number, Delivery_Status

Redundancy Analysis

Shipment information is stored separately from the Order table. This prevents repeated storage of tracking numbers and delivery statuses.

Data Anomalies

- **Insert Anomaly:** Shipment details can be added after an order is confirmed.
- **Update Anomaly:** Delivery status can be updated independently.
- **Delete Anomaly:** Deleting shipment information does not delete the order details.

9. ADMIN TABLE:

Functional Dependency

Admin_ID → Admin_Name, Email, Password

Redundancy Analysis

Administrator information is stored separately in the Admin table. This avoids repeating administrator login details in other tables.

Data Anomalies

- **Insert Anomaly:** A new administrator can be added independently.
- **Update Anomaly:** Administrator details can be updated in one place.
- **Delete Anomaly:** Removing an administrator does not affect customer, product, or order records.

CONCLUSION

The **Titan Eye Eyewear E-Commerce Database System** is designed using appropriate functional dependencies and normalized tables. Each entity stores only relevant information related to customers, products, categories, brands, orders, order items, payments, shipments, and administrators. The use of primary keys and foreign keys reduces data redundancy and prevents insert, update, and delete anomalies. This database design improves data consistency, maintains data integrity, and supports efficient management of the Titan Eye Eyewear e-commerce system. The functional dependencies are based on the primary keys and attributes identified in the Entity Analysis.